

COMP1002 Final Assignment

ZipRide Dispatch System: Data Structures and Algorithms for Efficient Ride-Hailing

S1 2026 @ EECMS, Curtin University

20th April, 2026

1. Introduction

ZipRide is a fast-growing ride-hailing company operating across a busy metropolitan area. Every minute, the ZipRide Dispatch System must match passengers to the nearest available driver, maintain up-to-date passenger and driver data, prioritise and dispatch the highest-priority pickup requests, and produce dispatch performance reports. All of this must be achieved with minimal latency.

In this assignment you will design and implement a backend ZipRide Dispatch System using fundamental data structures and algorithms including Graph, Hash Table, Heap, and Sorting. These four modules form an integrated pipeline supporting real-time decision-making and post-operation analysis.

Module	Name	Role in ZipRide
1	Graph-based Route Planning	Models the city road network; finds the fastest driving route between any two locations
2	Hash-Based Passenger and Driver Lookup	Stores passenger and driver data for fast retrieval
3	Heap-based Pickup Scheduling	Dispatches pickup requests by computing priority using data from Modules 1 and 2.
4	Sorting Pickup Records	Sorts pickup records for end-of-day reporting.

2. Learning Objectives

This assignment aims to improve students' understanding and application of fundamental data structures and algorithms in solving real-world problems.

- Implement and analyse an adjacency-list graph with BFS, DFS, and Dijkstra's shortest-path algorithm.
- Design a hash table with collision handling suitable for at least 40 records.
- Implement a max-heap scheduler that combines passenger tier and estimated driving time from earlier modules.
- Implement merge sort and quick sort; benchmark and analyse performance across varied input conditions.
- Integrate all four modules into a coherent, testable system with clean data flow.

3. Module 1: Graph-Based Route Planning

3.1. Problem Description

ZipRide operates across a city whose road network is modelled as a **weighted, undirected** graph.

- **Nodes** represent locations (e.g., CBD, Airport, University, SuburbNorth, SuburbSouth, ShoppingMall, Hospital, IndustrialPark, etc.). You can define your nodes.
- **Edges** represent roads between locations.
- **Weights** are driving time in minutes.

3.2. Tasks

3.2.1. Graph Construction

- Implement a Graph class using an **adjacency list** (no built-in graph libraries).
- Dynamic insertion of locations (nodes) and roads (weighted undirected edges).
- Ensure undirected symmetry ($u \leftrightarrow v$ with same weight)

3.2.2. Algorithms to Implement (no built-in graph libraries)

- Breadth-First Search (BFS): Input: source location (e.g. CBD). Output: all reachable locations grouped by level (Level 0, Level 1, ...).
- Depth-First Search (DFS): Input: source location. Output: whether the graph contains a cycle; if found, show the locations involved.
- Shortest Path Algorithm (Dijkstra Algorithm (Cormen et al., 2022)): Input: source location. Output: shortest driving time and path from the source to destination.

3.2.3. Test Graph Specification

- At least 8 nodes.
- At least 10–12 weighted edges with driving times in minutes.
- At least one cycle and one isolated node to test robustness.

3.2.4. Expected Output

- Textual/visual graph structure.
- BFS output with nodes grouped by level.
- DFS output with cycle detection result and cycle members if a cycle exists.
- Dijkstra output: shortest path and driving time.

4. Module 2: Hash-Based Passenger and Driver Lookup

4.1. Problem Description

The ZipRide Dispatch System must efficiently retrieve both passenger and driver information in real time to support matching and dispatch decisions. A hash table is used to store and retrieve passenger and driver data efficiently. When a pickup request is received, the system needs immediate access to passenger attributes (e.g., membership tier) as well as driver availability and location.

4.2. Tasks

4.2.1 Hash Table Design and Implementation

- Use either chaining (linked lists) or open addressing (linear probing). Justify your choice in the report.
- Implement a modulo-based hash function. Explain your parameter choices (table size, prime selection) and why it reduces clustering.
- Define your passenger record with fields: PassengerID (integer), Name, PickupLocation, MembershipTier (1–5, where 1 = Platinum / highest); you can define any new fields in addition to the given ones if necessary
- Define your driver record with fields: DriverID (integer), Name, CurrentLocation (node in Graph), AvailabilityStatus (Available, Busy, Offline); you can define any new fields in addition to the given ones if necessary

4.2.2 Core Operations

- Insert(record): Add a new passenger or driver record into the hash table with $O(1)$ expected time complexity; handle duplicates (update or reject with message).
- Search(passengerID/DriverID): $O(1)$ expected time complexity; return the full passenger/driver record or a not-found message.
- Delete(passengerID/DriverID): $O(1)$ expected time complexity; confirm deletion and handle missing keys gracefully.

4.2.3 Collision Handling

- Demonstrate at least one collision scenario and show how it is resolved (probe sequence or chain state).
- Track and print the load factor after every 10 inserts. Discuss (and optionally implement) resizing when load factor exceeds a threshold (e.g., 0.7).
- Explain worst-case behaviours.

4.2.4 Validation

- Validate inputs (e.g., MembershipTier 1–5 only; non-empty names; IDs unique/integer, driver status).
- Ensure robustness against out-of-range indices and null references.

4.2.5 Testing Requirements

- Insert at least 20 passenger records (include diverse MembershipTier levels and PickupLocation) and 20 driver records (include diverse CurrentLocation and AvailabilityStatus)
- Demonstrate successful searches (hits) and misses, demonstrate deletions and their effect on subsequent searches.
- Provide operation counts or timing snapshots to support $O(1)$ expected performance.

4.2.6 Expected Output:

- Log of inserts, searches, and deletes (concise, readable format).
- Explicit collision example(s) with intermediate states (e.g., probe indices or chain contents).
- Summary of load factor behaviour and performance analysis.

4.2.7 Deliverables for Module 2

- Source code for hash table (with comments and clear function headers).

- Test driver demonstrating inserts, searches, deletes, and collision handling.
- Short written justification of hash strategy, table size, and hash function.

5. Module 3: Heap-Based Pickup Scheduling

5.1. Problem Description

Dispatching a pickup depends not only on passenger priority but also on driver availability and proximity. When a pickup request is received, the system must determine which available driver should be assigned and which passenger request should be served first. This module implements a priority-based scheduling system using a heap, integrating data from both the Hash Table (Module 2) and the Graph (Module 1). The goal is to minimise passenger waiting time while prioritising high-tier customers.

For each incoming pickup request:

- 1) Retrieve all drivers with AvailabilityStatus = “Available” from the driver hash table (Module 2).
- 2) For each available driver: Use the Graph (Module 1) to compute the shortest EstimatedPickupTime (T) from that driver’s CurrentLocation to the passenger’s PickupLocation.
- 3) Select the driver with the minimum T (nearest driver).
- 4) Use this T value in the following priority calculation.

Each pickup request is evaluated using a priority function that combines **passenger importance** and **driver proximity**:

Priority formula:

$$\text{Priority} = (6 - M) + 1000 / T$$

Where:

- M = MembershipTier (retrieved from Hash Table in Module 2; 1 = highest priority, 5 = lowest priority)
- T = EstimatedPickupTime from the driver’s CurrentLocation to the passenger’s PickupLocation (computed using Dijkstra’s algorithm from Module 1); consider the special case where CurrentLocation is the same as the PickupLocation.

Higher priority values indicate more urgent and efficient pickups and are dispatched first.

5.2. Tasks

5.2.1 Heap Implementation (array-based binary heap; no built-in priority queues)

- Choose Max Heap (recommended) and justify; alternatively, use Min Heap with inverted keys. Justify the choice of max vs min heap in the report.
- Implement:
 - (i) insert(request): add a pickup request and percolate up to restore the heap property. Print the heap array after every insert.
 - (ii) peek(): return the highest-priority request without removing it.

- (iii) `extract_priority()`: remove and return the highest-priority request; percolate down to restore the heap property. Print the heap array after every extraction.

5.2.2 Edge Cases

- If `MembershipTier` changes, request is reinserted or updated.
- If driver becomes busy, removed from the available pool.
- If no drivers are available, request is rejected or queued.
- Handle invalid inputs (e.g., `PassengerID`, `MembershipTier`, etc.) and invalid `EstimatedPickupTime` gracefully.

5.2.3 Test Run

- The heap array is printed after each insert and `extract_priority`.
- At least 10 insert operations and 5 extractions are demonstrated.

5.2.4 Expected Output

- Sequence of heap states after operations (readable array/tree printout).
- Log of priority computations and assigned drivers.
- Correct dispatch order: prioritise nearest-driver + high-tier passengers.

5.2.5 Deliverables for Module 3

- Heap source code with comments.
- Test driver producing the required logs and traces.
- Written explanation of driver selection strategy and update handling.

6. Module 4: Sorting Pickup Requests

6.1. Problem Description

To optimise the dispatch process, the pickup requests should be sorted and reported at the end of the day. To achieve this, you will implement and evaluate two comparison-based sorting algorithms: Merge Sort and Quick Sort.

6.2. Tasks

6.2.1 Algorithms to Implement (no use of built-in sorting algorithms)

- Merge sort (top-down or bottom-up; justify the choice in the report).
- Quick sort (choose and justify your pivot strategy; justify and discuss its effect).
- Sort by `EstimatedPickupTime T` (minutes, ascending).

6.2.2 Dataset Sizes and Input Conditions

- Generate and sort datasets of 100, 500, and 1,000 pickup requests. This should be implemented in an integrated approach: compute `T` using the shortest path results from previous modules.
- For each size, test three conditions: random, nearly sorted ($\leq 10\%$ of records displaced), and reversed.
- Use a fixed random seed for reproducibility.

6.2.3 Analysis

- Present a table summarising times (can use `timeit` python library) and operation counts if collected.
- Write a brief analysis: where does each algorithm excel or degrade, and why (pivot choice, recursion depth, cache effects)?

6.2.4 Expected Outputs

- Correctly sorted lists
- Timing table (and, if available, plots) comparing merge sort and quick sort across input sizes and conditions.
- Short written reflection on algorithm choice under different scenarios.

6.2.5 Deliverables for Module 4

- Source code for merge sort and quick sort.
- Benchmark script and results (tables and optional plots).
- Brief write-up on stability and pivot strategy.

7. Submission Guidelines

7.1 Submission Deadline: **28 May Thursday 23:59 (AWST)**. An additional 7 days for students with the Curtin Access Plan (CAP).

7.2 Blackboard Submission: You must submit via **both** links. Submitting to only one link is considered an incomplete submission.

Link	What to submit
Turnitin link	Technical report in PDF format only. The Declaration of Originality must be included as the first page of the report.
Assignment link	Single ZIP file named COMP1002_assignment_<studentID>.zip, containing: • All source code (Python or Java) • README.md with module-wise run/test instructions • Sample input and output files • Technical report (same PDF as submitted to Turnitin)

7.3 Source Code Requirements

- Python or Java only. You may reuse code from practical submissions (self-cite in comments). For the shortest path algorithm, cite your algorithm reference.
- No built-in data-structure libraries for core logic (e.g., no dict/Map, no heapq/PriorityQueue, no sorted()/Arrays.sort() for sorting, etc). **A 100% penalty applies per module where a built-in is used.**
- No AI-generated code. **A 100% penalty applies to any affected section if detected.**
- All code must be modular with meaningful variable and function names.
- Include inline comments and a header block (author, date, description) in every file.
- Include error and exception handling in every module.

7.4 Technical Report Requirements: Submit a PDF report with clear section headers, numbered pages, and consistent formatting:

- 1) Overview: system purpose and modular design.
- 2) Data structures used: justification for graph, hash table, heap, and sorting algorithms.
- 3) Module integration: include the UML data-flow diagram (refer to Unit Materials > Links and Resources > UML Style Guide).
- 4) Algorithm complexity: time and space analysis for each module.
- 5) Sample output: screenshots or log excerpts per module.
- 6) Reflection: key learnings and design trade-offs.

- 7) Limitations and assumptions: known issues and simplifications.
- 8) Generative AI Declaration: state which AI tools, if any, were used for learning support (e.g. concept clarification). AI-generated code or report text is not permitted and will be penalised.
- 9) References: cite all sources used following the [Chicago 17th Author-Date](#). Do not include self-cited code.

7.5 Code Demonstration

- Demo Date: **2 June – 12 June 2026**
- Book a **20-minute** slot on Blackboard (link and venue details will be posted).
- Be prepared to run each module live (in person/online), explain code structure, and answer questions.
- Demonstration is **compulsory** to meet the passing criteria; non-attendance will result in **ZERO** marks for the assignment.
- This assignment is worth **40 points** and contributes 40% to the final grade.

7.6 Academic Integrity

- All work must be your own. You may discuss ideas with peers but must not share code.
- Cite all algorithmic references used (e.g., Dijkstra Algorithm).
- Any evidence of plagiarism or unauthorised code generation will be reported and penalised.
- For support, post on **Piazza** or contact unit coordinator, lecturer or tutors.

8. Marking Criteria (Total: 40 marks)

Criteria Details		Marks
Module 1 — Graph (8 marks)		
Graph construction and adjacency list design	Correct dynamic insertion; undirected symmetry enforced	2
BFS	Correct reachable nodes and level grouping	2
DFS	Correct cycle detection and reporting	2
Dijkstra	From-scratch implementation; accurate path and estimated driving time output	2
Module 2 — Hash table (6 marks)		
Insert, search, delete	All three operations implemented and correct	3
Collision handling	At least one collision demonstrated with intermediate state	1
Hash function and scalability	Effective design; handles 40+ records without degradation	2

Module 3 — Heap scheduling (6 marks)		
Insert and extract operations	Correct implementation; heap property maintained throughout	2
Priority score calculation	Correct use of MembershipTier (M) from Module 2 and EstimatedPickupTime (T) from Module 1	2
Heap state tracing/logging	Array printed after every insert and extraction	2
Module 4 — Sorting (6 marks)		
Correctness	Merge sort and quick sort produce correct sorted output	3
Performance analysis	Timing table across sizes and conditions; written analysis	3
Technical report (8 marks)		
Overview and justification	Clear system purpose; justified data-structure choices	2
UML / flowchart	Module interactions and data flow clearly shown	2
Complexity analysis and reflection	Time and space analysis; performance trade-offs discussed	2
Sample outputs	Screenshots/logs; assumptions and limitations documented	2
Code quality and compliance (6 marks)		
Modular, readable, documented code	No built-ins for core logic; clean structure	2
Output traces and edge cases	Sample inputs tested; edge cases demonstrated	2
Error handling	Exceptions handled: input validation in all modules	2

This marks the end of the assignment specification. Good luck and have fun!

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms*. MIT press.